# Indian National AI Olympiad - 2026 Stage 1 Alternate Test

February, 28, 2026

## Section A1

## Cracking the Imperial Glyphs [3 pts]

Naruto, a time-travelling linguist, lands in an ancient palace, where people communicate in a mysterious language. In this language, the order of symbols matters and there may not be a one-to-one correspondence with English words. He begins learning the language by matching illustrated pictures with their meanings.

| English | Imperial Glyphs |
|---|---|
| This is a beautiful apple. | 這美蘋果 |
| This is a very beautiful apple. | 這很美蘋果 |
| This is an apple on a tree. | 這蘋果樹 |
| They serve green apples in the palace. | 服綠蘋果宮 |
| Does the tree make the apple beautiful? | 樹造蘋果美? |
| There is a palace on the black mountain. | 這黑山宮 |
| The mountain paints the palace black. | 山塗宮黑 |
| This is a sweet apple. | 這甜蘋果 |

**Output**

Naruto now wants to compliment the palace gardener by correctly saying:

*"The soil makes the apple sweet."*

Which imperial glyph message should he use?

**Options:**

| Option | Imperial Glyphs |
|---|---|
| A | 山造蘋果甜 |
| B | 土造蘋果很甜 |
| C | 蘋果造土甜 |
| D | 土塗宮甜 |
| E | 土造蘋果甜 |
| F | 蘋果造甜山 |

## Filtering Fakes with Faulty Oracle [3 pts]

You are building a fake-news classifier that operates on a pair consisting of a French news headline and its accompanying article body. You plan to use a small offline pretrained text-only generative model (e.g., Mistral 7B, LLaMA 2, Phi-2) augmented with real-time web retrieval as oracle to extract features, and train a supervised classifier on top of these features using provided true labels (Fake vs. True).
The feature-extraction prompt produces the following table.

| ID | Feature Name | Prompt to the Model |
|---|---|---|
| $F_1$ | Character count | "Count the number of characters in the headline." |
| $F_2$ | Estimated publication year | "Guess the publication year of this headline." |
| $F_3$ | Political leaning | "Classify the political leaning of the outlet that likely published this headline as: left, centre, or right." |
| $F_4$ | Article–headline consistency | "Does the evidence presented in the article body support the headline? (yes/no)" |
| $F_5$ | Language identification | "Identify the language of the headline and return the ISO-639-1 two-letter code only (e.g., `fr`,). If uncertain, return `unk`" |
| $F_6$ | Named outlet prediction | "Which French news outlet likely published this headline?" |
| $F_7$ | Topic classification | "Classify topic: science, sports, politics, literature, finance, other." |
| $F_8$ | Quote/attribution presence | "Does the headline contain a quote or attribution? (yes/no)" |

**Output**

Which of the following feature subsets are structurally the most unreliable when produced by this model under the described setting and should be excluded?

**Options:**

A. $\{F_1, F_2, F_4\}$

B. $\{F_2, F_3, F_4\}$

C. $\{F_1, F_4, F_6\}$

D. $\{F_3, F_4, F_5\}$

E. $\{F_1, F_3, F_4\}$

F. $\{F_1, F_3, F_6\}$

## Smart Professor Search [3 pts]

You are a student intern helping to improve your university's faculty directory search system. Each professor profile has the format: (*Department, Title, Name*), e.g., (`CS`, `Dr.`, `Alice Johnson`). You plan to create a custom profile similarity function by building on two standard string comparison measures:

- **Edit Distance:**

  $\mathrm{ED}(x, y)$ = minimum number of single-character insertions, deletions, or substitutions to transform $x$ into $y$.

- **Normalized Edit Distance:**
  Normalizing edit distance by the length of the longer string yields:

$$\mathrm{NED}(x, y) = 1 - \frac{\mathrm{ED}(x, y)}{\max(|x|, |y|)} \in [0, 1]$$

  where $|x|$ denotes the number of characters in string $x$.

**Design Rules:**
Your custom similarity function $\mathrm{Sim}(p, q)$ must satisfy the following requirements:

1. $\mathrm{Sim}(p, q) = 1$ if and only if all fields match exactly.
2. $\mathrm{Sim}(p, q) = 0$ if department codes differ.
3. $\mathrm{Sim}(p, q) = \mathrm{Sim}(q, p)$, i.e., it is symmetric.
4. When departments match, name mismatches reduce the score more than title mismatches.
5. $\mathrm{Sim}(p, q) \in [0, 1]$ for all $p, q$.

Let each professor profile be written as $p = (d, t, n)$ and $q = (d', t', n')$, where the three fields denote the department, title, and name in that order. Let $\mathbf{1}_{x=x'}$ denote the indicator function that equals 1 when $x = x'$ and 0 otherwise.

**Output**

Which among the following functions is a good choice for the custom similarity function $\mathrm{Sim}(p, q)$?

**Options:**

A. $\mathbf{1}_{d=d'} \cdot \dfrac{\mathrm{NED}(n, n') + \mathrm{NED}(t, t')}{2}$

B. $\mathrm{NED}(d, d') \cdot \exp\Big( -2\big(1 - \mathrm{NED}(n, n')\big) - \big(1 - \mathrm{NED}(t, t')\big)\Big)$

C. $\mathbf{1}_{d=d'} \cdot \exp\Big( -2\big(1 - \mathrm{NED}(n, n')\big) - \big(1 - \mathrm{NED}(t, t')\big)\Big)$

D. $\mathbf{1}_{d=d'} \cdot \exp\Big( -\big(1 - \mathrm{NED}(n, n')\big) - 2\big(1 - \mathrm{NED}(t, t')\big)\Big)$

E. $\mathbf{1}_{d=d'} \cdot \Big(2 \cdot \mathrm{NED}(n, n') + \mathrm{NED}(t, t')\Big)$

F. $\mathbf{1}_{d=d'} \cdot \big(\exp\big(-2\big(1 - \mathrm{NED}(n, n')\big)\big) + \exp\big(-\big(1 - \mathrm{NED}(t, t')\big)\big)\big)$

## Recursive V-67 Outbreak [3 pts]

A highly contagious virus, $V$-67, is spreading in a population. There are three groups of people:

- $N_t$: number of normal infectious individuals at the start of day $t$,
- $M_t$: number of mutant infectious individuals at the start of day $t$,
- $R_t$: number of recovered individuals who do not infect others and cannot be infected again.

**Note:** Treat all counts as real-valued with no intermediate rounding.

Each day proceeds in the following order:

1. **New infections.** Each normal infectious person infects 2 new people. The mutant strain is evolving: each mutant infectious person infects $(2 + 4t)$ new people on day $t$, becoming progressively more contagious over time. The total number of new infections on day $t$ is:
$$X_t = 2N_t + (2 + 4t)M_t.$$
2. **Fate of the newly infected.** Of these $X_t$ newly infected people, all enter the infectious pool:
$$\frac{3}{4}X_t \text{ become normal infectious,} \qquad \frac{1}{4}X_t \text{ become mutant infectious.}$$
3. **Recovery of existing infectious individuals.** At the end of the day, half of the individuals who were infectious (of both types) at the *start* of the day recover and join $R$:
$$\frac{1}{2}N_t \;\to\; R, \qquad \frac{1}{2}M_t \;\to\; R.$$
Newly infected individuals do not recover on the same day they are infected.

**Output**

Given that the outbreak starts on Day 0 with $N_0 = 8$, $M_0 = 8$, $R_0 = 0$, what is the total number of recovered individuals at the start of Day 3, that is, compute $R_3$?

**Options:**

A. 72

B. 78

C. 90

D. 102

E. 138

F. 144

## Pikachu's Path to Victory [3 pts]

Pikachu is preparing for his ultimate challenge on a battlefield made of **Thunder Pads** arranged in a $4 \times 4$ grid as shown in Figure 1. Each pad crackles with electric energy that Pikachu can absorb on his journey to become victorious. He starts on the pad at the top-left $(0, 0)$ and must reach the final pad in the bottom-right corner $(3, 3)$.

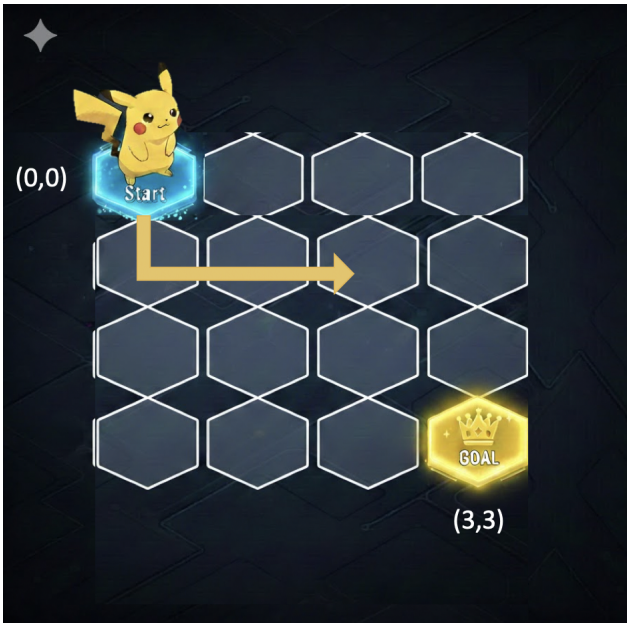



Figure 1: Pikachu's Thunder Pad battlefield.

**Rules of Movement:**

- **Thunder Leap:** Pikachu moves exactly like a knight in chess: two squares in one cardinal direction and one square perpendicular to it.
- **Spark Charge:** Every time Pikachu lands on a pad, he *activates* it and absorbs its energy.
- **One-Time Power:** Once a pad has been activated, Pikachu *cannot* land on it again.

**Output**

What is the **maximum number of pads** Pikachu can activate (including both the start and the final pad) on a valid path from $(0, 0)$ to $(3, 3)$?

**Options:**

A) 10

B) 12

C) 13

D) 14

E) 15

F) 16

# Section A2

# The Phulera Godown Showdown [3 pts]

In the village of Phulera, the four biggest rice fields are denoted as $R_i(x_i, y_i)$ for $i = 1, 2, 3, 4$, and are located at the coordinates:

$$R_1(40,\ 180), \quad R_2(120,\ 80), \quad R_3(220,\ 120), \quad R_4(80,\ 20).$$

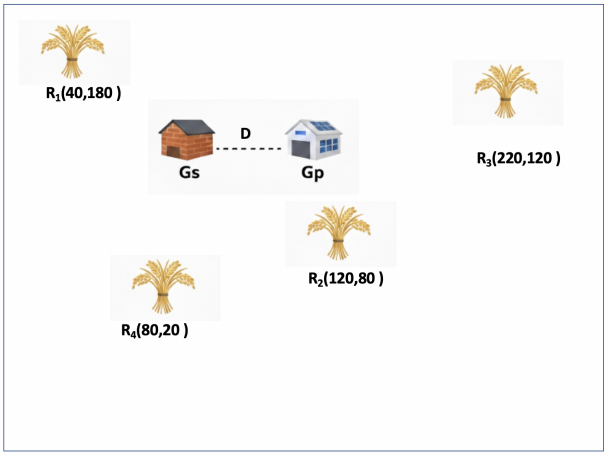



Figure 2: Phulera's rice fields and godown placement. Not to scale.

Sachivji argues that because the village has criss-cross perpendicular roads, travel cost is best modeled using Manhattan ($L_1$) distance. He proposes placing the village godown at

$$G_S = (x_S, y_S) = \arg\min_{(x,y)} \sum_{i=1}^{4} \big(|x - x_i| + |y - y_i|\big).$$

Pradhanji, to showcase his advanced thinking, proposes modeling the placement cost using the **Itakura–Saito (IS) distance**. The IS distance from a field coordinate $p > 0$ to a candidate location coordinate $u > 0$ is defined as:

$$\mathrm{IS}(p,\ u) = \frac{p}{u} - \ln\frac{p}{u} - 1 \ \geq\ 0.$$

Pradhanji proposes placing the godown at

$$G_P = (x_P, y_P) = \arg\min_{(u,v)} \sum_{i=1}^{4} \Big[\mathrm{IS}(x_i,\ u) + \mathrm{IS}(y_i,\ v)\Big].$$

**Output**

What is $\lfloor D \rfloor$ if $D$ is the Euclidean distance between Pradhanji's and Sachivji's godowns, i.e.,

$$D = \sqrt{(x_S - x_P)^2 + (y_S - y_P)^2}\,?$$

**Note:** If multiple points satisfy Pradhanji or Sachivji's conditions, they choose the arithmetic mean of the possible coordinate ranges to determine $x_s$ and $y_s$.

## The Blind Board Gambit [3 pts]

In a 3-board team chess match, Team Gukesh and Team Anand each have three players. Each team secretly and independently assigns one player to each board, and players on the same board compete against each other. Each game is played until one player wins (no draws). A team wins the match if it wins at least 2 of the 3 board games. Results across boards are independent.

**Ratings and Win Probability:** When a player with rating $R$ plays against a player with rating $S$, the probability that the $R$-rated player wins is

$$P(R \text{ beats } S) = \frac{1}{1 + 10^{(S-R)/400}}.$$

The teams have the following ratings:

- **Team Gukesh:** $A = 2100$, $B = 1800$, $C = 1600$
- **Team Anand:** $X = 2000$, $Y = 1700$, $Z = 1500$

**Scenario:** A spy tips off Anand that Gukesh's weakest player $C$ will *not* play on Board 3. This means Gukesh will use one of the following three assignments, each equally likely:

$$(B,\, C,\, A), \quad (C,\, A,\, B), \quad (C,\, B,\, A).$$

Anand selects the assignment of $(X, Y, Z)$ that maximises his expected win probability over this distribution *without* knowledge of Gukesh's assignment. Gukesh's actual assignment is $(C, A, B)$ to Boards $(1, 2, 3)$.

**Output**

Let $p$ be the probability that Team Anand wins the match under this scenario. Compute $\lfloor 100p \rfloor$.

## The Crude Curve Forecast [3 pts]

Prof. Clairvoyant, an oil market analyst, models crude prices by decomposing them into multiple effects: a baseline level, a long-term linear trend, a sudden price drop due to a geopolitical shock, a gradual exponential recovery following a supply agreement, a demand-driven downward trend that sets in later, and seasonal fluctuations. Using observations of the monthly average crude oil price per barrel $P(t)$, he arrives at the following continuous time model:

$$P(t) = A + Bt + C \cdot \left(1 - e^{-(t-\tau_C)}\right) \cdot h(t - \tau_C) + D \cdot h(t - \tau_D) + F \cdot (t - \tau_F) \cdot h(t - \tau_F) + E \cos\left(\frac{2\pi t}{\tau_E}\right)$$

where $t$ is time in months from Jan 2020 ($t = 0$), the parameters $A, B, C, D, E, F, \tau_C, \tau_D, \tau_E, \tau_F$ are all unknown, and $h(t)$ is the Heaviside step function defined as

$$h(t) = \begin{cases} 0, & t < 0 \\ 1, & t \geq 0. \end{cases}$$

Figure 3 shows $P(t)$ for $0 \leq t \leq 60$ months. The graph is drawn to scale with key values readable at strategic time points.

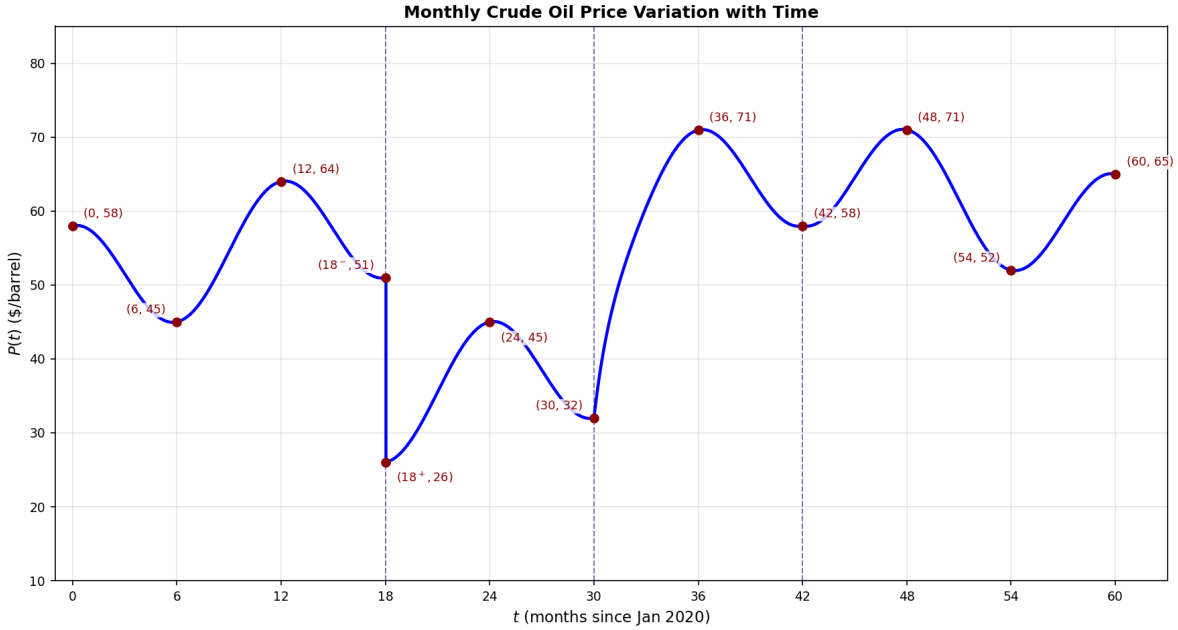



*Figure 3: Monthly crude oil price variation with time.*

**Output**

Using the price model and the above graph, estimate $\lfloor P(75) \rfloor$ where $P(75)$ is the price at $t = 75$ months.

## The Catalyst Constraint [3 pts]

You are a pharmaceutical process engineer optimising the production of drug compound D. Two parallel synthesis pathways can produce D:

$$\textbf{Pathway A:} \quad A \xrightarrow{C} D \qquad\qquad \textbf{Pathway B:} \quad B \xrightarrow{C} D$$

where $A$ and $B$ are different substrate molecules, and $C$ is a precious catalyst that must be shared between both pathways.

**Reaction Kinetics:** The yield of compound D from each pathway follows quadratic kinetics due to catalyst inhibition effects:

$$\textbf{Pathway A:} \quad d_A = 10 \cdot c_A - a \cdot c_A^2, \qquad \textbf{Pathway B:} \quad d_B = 20 \cdot c_B - b \cdot c_B^2,$$

where:

- $a$, $b$ are moles of substrates $A$ and $B$ available,
- $c_A$, $c_B$ are moles of catalyst $C$ allocated to each pathway,
- $d_A$, $d_B$ are moles of product $D$ from each pathway.

**Output**

Your production batch has the following resources: $a = 2$ moles of substrate $A$, $b = 3$ moles of substrate $B$, and 5 moles of catalyst $C$.

What is the maximum total yield of compound D (in moles) that you can produce by optimally allocating catalyst $C$ between the two pathways? Report $\lfloor d_A + d_B \rfloor$ rounded down to the nearest integer.

## The Pit-Stop Paradox [3 pts]

It is the final 10 laps of the Monaco Grand Prix. Max Verstappen (Driver V) is leading Lewis Hamilton (Driver H), but Verstappen's tyres are critically worn while Hamilton's are in better condition. Both teams must decide: pit for fresh tyres or stay out.

**Strategy Probabilities (Safety Car Dependent):** Each driver independently chooses whether to pit in the final 10 laps. A Safety Car (SC) may be deployed following an on-track incident, which significantly alters the cost of pitting. Historical data and team behaviour suggest the following probabilities:

| **Strategy** | **P(Strategy \| No SC)** | **P(Strategy \| SC)** |
|---|---|---|
| Neither pits | 0.50 | 0.05 |
| Only V pits | 0.25 | 0.10 |
| Only H pits | 0.10 | 0.05 |
| Both pit | 0.15 | 0.80 |

**Race Outcome Probabilities:** Race simulations estimate the probability that Hamilton wins under each scenario:

| **Strategy** | **P(H wins \| Strategy, No SC)** | **P(H wins \| Strategy, SC)** |
|---|---|---|
| Neither pits | 0.20 | 0.40 |
| Only V pits | 0.85 | 0.15 |
| Only H pits | 0.10 | 0.90 |
| Both pit | 0.50 | 0.55 |

**Output**

You arrive after the checkered flag and learn that Hamilton won the race. You do not know whether a Safety Car was deployed or which pit strategies were chosen.

Given that there was a prior 40% chance of a Safety Car deployment based on historical incident data, what is your estimate of the probability that neither driver pitted? Report $\lfloor 100 \times P(\text{neither pitted} \mid H \text{ wins}) \rfloor$.

# Section B1

# Code Triage: Critical Classification Challenge

**Background: Multi-Class Classification**

In a multi-class classification problem, the goal is to assign instances to one of several possible classes. The performance of the classifier is evaluated using the following metrics.

**Confusion Matrix:** The confusion matrix for a 3-class problem is structured as:

$$\begin{bmatrix} \text{TP}_{C_1} & F_{C_1,C_2} & F_{C_1,C_3} \\ F_{C_2,C_1} & \text{TP}_{C_2} & F_{C_2,C_3} \\ F_{C_3,C_1} & F_{C_3,C_2} & \text{TP}_{C_3} \end{bmatrix}$$

where:

- $\text{TP}_{C_i}$: True positives for class $C_i$ (instances correctly classified as $C_i$)
- $F_{C_i,C_j}$: Instances of class $C_j$ incorrectly classified as $C_i$

**Binary Classification Metrics:** For multi-class problems, we can treat one class as "positive" and all others as "negative" to compute binary metrics.
**Sensitivity (True Positive Rate):** For class $C_i$ as positive, sensitivity measures the proportion of actual $C_i$ instances correctly identified:

$$\text{Sensitivity}_i = \frac{\text{TP}_{C_i}}{\text{TP}_{C_i} + \sum_{j \neq i} F_{C_j,C_i}}$$

**Specificity (True Negative Rate):** For class $C_i$ as positive, specificity measures the proportion of actual non-$C_i$ instances correctly identified as non-$C_i$:

$$\text{Specificity}_i = \frac{\text{TN}_{C_i}}{\text{TN}_{C_i} + \sum_{j \neq i} F_{C_i,C_j}}$$

where $\text{TN}_{C_i}$ is the number of instances correctly classified as not $C_i$.

**ROC Curve:** The Receiver Operating Characteristic (ROC) curve plots True Positive Rate (TPR = Sensitivity) against False Positive Rate (FPR = 1 - Specificity) at various decision thresholds.

## Problem Context

A hospital emergency department uses an automated triage system. A linear classifier analyses patient vital signs, lab results, and symptoms to produce a **risk score**. Two thresholds ($t_1 < t_2$) partition this score into three severity levels:

- **Class 1 — Outpatient:** Score $< t_1$
- **Class 2 — Regular Ward:** $t_1 \leq$ Score $< t_2$
- **Class 3 — ICU:** Score $\geq t_2$

The system was evaluated on **1000 patients** with the following **actual** class distribution:

Outpatient (C1): 50%  Regular Ward (C2): 35%  ICU (C3): 15%

The classifier achieves the following performance:

| Class | Sensitivity | Specificity |
|---|---|---|
| Class 1 (Outpatient) | 0.84 | 0.90 |
| Class 2 (Regular Ward) | 0.70 | 0.88 |
| Class 3 (ICU) | 0.80 | 0.89 |

Analysis of score distributions shows that egregious (two-class-skip) errors are rare:

- **10** actual ICU patients (Class 3) were misclassified as Outpatient (Class 1).
- **25** actual Outpatient patients (Class 1) were misclassified as ICU (Class 3).

## Questions

### Q1: False Alarms in ICU Prediction [1 pt]

**Output**

Among all patients the system flags for ICU care (predicted Class 3), what fraction $x$ do *not* actually require ICU care? Report your answer as a percentage rounded down: $\lfloor 100x \rfloor$.

### Q2: Inpatient Triaging [1 pt]

**Output**

Among patients who *actually* require hospitalisation (actual Class 2 or Class 3), what fraction $x$ were correctly identified as needing hospitalisation (i.e. predicted as Class 2 or Class 3 rather than sent home)? Report $\lfloor 100x \rfloor$.

### Q3: Optimal Operating Point for ICU Detection [2 pts]

For ICU detection (Class 3 as positive, Classes 1 and 2 as negative), varying $t_2$ traces the ROC curve in Figure 3. The **current operating point** corresponds to a specific threshold settings.

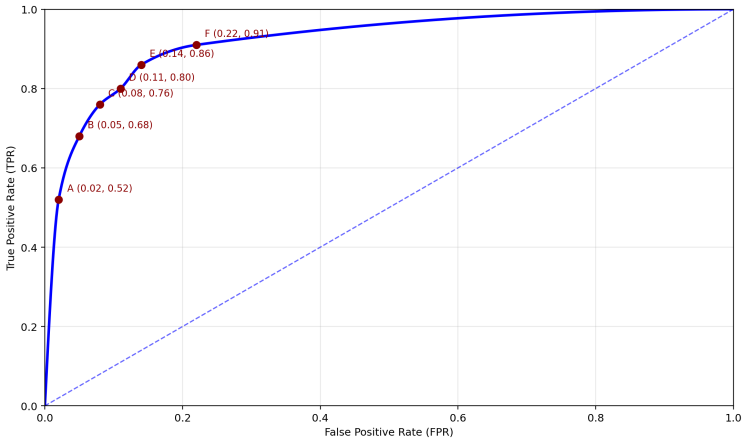



Figure 3: ROC curve for ICU detection with candidate operating points (A–F).

The medical team has quantified mis-classification costs for the binary ICU / non-ICU decision:

| | Pred. ICU | Pred. Non-ICU |
|---|---|---|
| **Actual ICU** | 0 | 8 (missed ICU) |
| **Actual Non-ICU** | 1 (unnecessary admission) | 0 |

**Output**

Which operating point minimises the expected misclassification cost per patient?

**Options:**

1. Point A (FPR $= 0.02$, TPR $= 0.52$)
2. Point B (FPR $= 0.05$, TPR $= 0.68$)
3. Point C (FPR $= 0.08$, TPR $= 0.76$)
4. Point D (FPR $= 0.11$, TPR $= 0.80$)
5. Point E (FPR $= 0.14$, TPR $= 0.86$)
6. Point F (FPR $= 0.22$, TPR $= 0.91$)

### Q4: ROC Curve — Effect of Score Adjustment [1 pt]

A researcher proposes augmenting the risk score using a rapid-diagnostic-test (RDT) biomarker for sepsis, $b \in \{0, 1\}$:

$$s_{\text{new}} \;=\; s_{\text{old}} \;+\; \delta\,(2b - 1), \qquad \delta = 1.5 > 0.$$

RDT-positive patients ($b = 1$) receive a score *increase* of $\delta$. It is known that the RDT has sensitivity 0.85 and specificity 0.88 for ICU-level illness.

Assume that the biomarker provides independent predictive information beyond the original score, i.e.

$$P(b \mid s_{\text{old}}, y) = P(b \mid y),$$

and that the augmented score $s_{\text{new}}$ is used directly for ranking.

**Output**

How does this transformation affect the ROC curve for ICU detection, assuming nothing else changes?

**Options:**

1. The curve remains unchanged.
2. The curve improves consistently (toward top-left).
3. The curve worsens consistently (away from top-left).
4. The curve improves only if $\delta$ exceeds the standard deviation of $s_{\text{old}}$.
5. The curve improves only at high-FPR operating points (right side, lower specificity).
6. Cannot be determined without the full joint distribution of ($s_{\text{old}}$, $b$) per class.

### Q5: Diagnosing a ROC Curve Shift [1 pt]

Six months after deployment, the hospital's data science team recomputes the ROC curve for ICU detection on a new cohort of 1 000 patients from the same population. They observe that the new ROC curve lies **consistently above and to the left** of the original across all operating points.

**Output**

Which is the most likely explanation for this shift?

**Options:**

1. The ICU prevalence in the new cohort rose from 15% to 22%.
2. The threshold $t_2$ was lowered, flagging more patients as ICU.
3. Two new predictive features were added to the scoring function before re-evaluation.
4. All risk scores were transformed with a monotonic square-root function before applying thresholds.
5. The evaluation cohort was doubled to 2 000 patients, reducing sampling noise.
6. The new cohort excluded a random subset of ICU patients who passed away before the study.

**Q6: Comparing Classifier Architectures and Loss Functions [2 pts]**

You are the chief scientist for the hospital. Medical domain analysis shows that criteria for distinguishing Outpatient vs. Inpatient (Classes 2 and 3) differ from the criteria for Regular Ward vs. ICU (e.g., pain/infection markers vs. vital instability/organ function). You wish to design a new advanced classification system for the three-class triage problem but prefer to stick with linear models so that the decisions are interpretable for doctors and training data is limited. You are considering the following architectures and loss function:

**Architectures:**

- **X:** Single linear scorer with two thresholds $t_1 < t_2$.
- **Y:** Two binary classifiers in a cascade: first C1 vs {C2,C3}; if not C1, then C3 vs C2.
- **Z:** Three pairwise classifiers: (C1 vs. C2), (C2 vs. C3), (C3 vs. C1).

**Loss functions** (true label $y \in \{-1, +1\}$, predicted score $f(x)$):

- $L_{\text{SQ}}$: $(y - f(x))^2$
- $L_{\text{HG}}$: $\max(0,\ 1 - y\, f(x))$
- $L_{\text{LOG}}$: $\log(1 + e^{-y\, f(x)})$

**Output**

Consider the following claims and select **all** that are true. (More than one answer may be correct.)

**Options:**

1. **Claim 1:** Y and Z can produce predictions inconsistent with a strict severity ordering, requiring post-processing to assign a final class label.
2. **Claim 2.** Z achieves strictly greater classification accuracy than Y on any 3-class dataset, because three pairwise decision boundaries always span more classification regions than two hierarchical boundaries.
3. **Claim 3.** For Architecture X, $L_{\text{SQ}}$ is better suited than $L_{\text{HG}}$ at enforcing ordinal structure, but is more sensitive to outliers.
4. **Claim 4.** For Architecture Y, $L_{\text{SQ}}$ is more robust to outliers than $L_{\text{HG}}$.
5. **Claim 5.** For Architecture Y, minimising $L_{\text{LOG}}$ yields well-calibrated class probability outputs, whereas $L_{\text{HG}}$ does not.
6. None of the above hold true.

**Q7: Expected Misclassification Cost [2 pts]**

The hospital assigns misclassification costs as follows:

| | **Predicted C1** | **Predicted C2** | **Predicted C3** |
|---|---|---|---|
| **Actual C1** | 0 | 10 | 100 |
| **Actual C2** | 50 | 0 | 20 |
| **Actual C3** | 500 | 100 | 0 |

**Output**

Using the performance information of the current operating point (in Problem Context), compute the expected misclassification cost $x$ per patient averaged over all 1 000 patients. Report $\lfloor x \rfloor$.

## Section B2

## WonkaRank: The Golden Ticket Retrieval Challenge

**Background: Information Retrieval Metrics**

In information retrieval, documents are represented as vectors in a high-dimensional space and ranked by a score function to retrieve documents relevant to a user's need.

**Vector Representation:** Given a vocabulary $V = \{t_1, t_2, \ldots, t_m\}$ of $m$ terms, each document $D^{(j)}$ is a vector in $\mathbb{R}^m$:

$$D^{(j)} = (d_1^{(j)}, d_2^{(j)}, \ldots, d_m^{(j)})$$

where $d_i^{(j)}$ is the weight of term $t_i$ in document $D^{(j)}$.

**Relevance:** Each document has a true relevance $\text{Rel}(D^{(j)}) \in \mathbb{R}^+$, where higher values indicate greater relevance. A document is relevant if $\text{Rel}(D^{(j)}) > 0$.

**Ranking Function:** A score function $\text{Score}(D^{(j)}) \in \mathbb{R}$ assigns a score to each document. Documents are ranked in decreasing score order: $D^{(1)}, D^{(2)}, D^{(3)}, \ldots$ Ties are broken by **numeric Chocolate ID in ascending order** ($D_1 < D_2 < \cdots < D_{10}$).

**Retrieval Metrics:**
**(a) Precision and Recall:** Given the top $k$ results and $R$ total relevant documents:

$$\text{Recall@}k = \frac{\text{Number of relevant documents in top } k \text{ results}}{R}$$

$$\text{Precision@}k = \frac{\text{Number of relevant documents in top } k \text{ results}}{k}$$

where a document is relevant if $\text{Rel}(D) > 0$.

**(b) Discounted Cumulative Gain (DCG):** DCG measures ranking quality by considering both relevance values and document positions:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{Rel}(D^{(i)})}{\log_2(i+1)}$$

where $D^{(i)}$ is the document at position $i$. The logarithmic discount emphasises placing highly relevant documents early. Here, @$k$ indicates that DCG is computed using only the top $k$ ranked documents.

**(c) Position-Based Inspection (PI-Recall@$k$):** Users often do not examine all results due to attention or satisfaction decay. Let $p_i$ be the probability that a user inspects position $i$. Position-based Inspection Recall is:

$$\text{PI-Recall@}k = \frac{1}{R} \times \sum_{i=1}^{k} p_i \times \mathbf{1}[\text{Rel}(D^{(i)}) > 0]$$

where $\mathbf{1}[\cdot]$ is the indicator function (1 if true, 0 if false).

## Problem Context

Willy Wonka has hidden Golden Chocolate Factory Tour tickets inside a small number of premium chocolates sold across the city to celebrate the New Year. Each chocolate lists the quantities of four ingredients on its wrapper: Cocoa (C), Hazelnut (H), Caramel (K), and Mint (M).

Charlie wants to find tour tickets while minimising spending, so he designs ranking strategies that place chocolates with tickets early in his purchase order.

## Celebration Chocolate Collection

The city has **10** premium chocolates with the following ingredient and ticket information.

| Chocolate ID | C | H | K | M | Golden Ticket? | Relevance |
|---|---|---|---|---|---|---|
| $D_1$ | 1 | 1 | 0 | 0 | VIP Tour | 10 |
| $D_2$ | 1 | 1 | 1 | 1 | No Ticket | 0 |
| $D_3$ | 1 | 0 | 1 | 1 | No Ticket | 0 |
| $D_4$ | 0 | 1 | 1 | 1 | No Ticket | 0 |
| $D_5$ | 1 | 1 | 0 | 1 | Partial Tour | 3 |
| $D_6$ | 0 | 0 | 1 | 1 | No Ticket | 0 |
| $D_7$ | 1 | 0 | 0 | 0 | No Ticket | 0 |
| $D_8$ | 1 | 1 | 0 | 0 | Full Tour | 5 |
| $D_9$ | 0 | 1 | 1 | 0 | Partial Tour | 2 |
| $D_{10}$ | 0 | 1 | 0 | 0 | No Ticket | 0 |

A chocolate is considered relevant if it contains any tour ticket (Relevance $> 0$).

## Questions:

### Q1: DCG@5 Calculation [1 pt]

Charlie purchases the following 5 chocolates in order: [$D_5$, $D_3$, $D_1$, $D_9$, $D_8$].

**Output**

Compute $\lfloor \text{DCG@5} \rfloor$ for this ranking.

### Q2: Comparison of Ranking Strategies [1 pt]

Charlie compares two scoring-based strategies. Both rank chocolates by decreasing score with ties broken by numeric Chocolate ID (ascending).

**Strategy A:**

$$\text{Score}_A(D) = \begin{cases} 2 & \text{if chocolate contains both Cocoa and Hazelnut} \\ 0 & \text{otherwise} \end{cases}$$

**Strategy B:**

$$\text{Score}_B(D) = \begin{cases} 2 & \text{if chocolate contains both Cocoa and Hazelnut} \\ 1 & \text{if chocolate contains exactly one of Cocoa or Hazelnut} \\ 0 & \text{if chocolate contains neither} \end{cases}$$

**Output**

Which statement best describes Strategy B compared to Strategy A at $k = 8$? (Ties are broken by numeric Chocolate ID in ascending order.)

**Options:**

1. Strategy B improves recall but reduces precision.
2. Strategy B improves precision but reduces recall.
3. Strategy B maintains recall but reduces precision.
4. Strategy B improves both precision and recall.
5. Both strategies produce identical recall.
6. None of the above.

**Q3: Maximising DCG [2 pts]**

Charlie has identified four relevant chocolates $\{D_1, D_5, D_8, D_9\}$ and wishes to rank them in an order that maximises DCG@4.

**Output**

Which ordering of $\{D_1, D_5, D_8, D_9\}$ achieves the highest DCG@4?

**Options:**

1. $[D_1, D_5, D_8, D_9]$
2. $[D_8, D_1, D_5, D_9]$
3. $[D_1, D_8, D_5, D_9]$
4. $[D_5, D_9, D_1, D_8]$
5. $[D_1, D_8, D_9, D_5]$
6. $[D_9, D_5, D_8, D_1]$

**Q4: Optimal Scoring Function [3 pts]**

Let each chocolate $D$ have ingredient vector $(d_C, d_H, d_K, d_M)$ and let $W = (w_C, w_H, w_K, w_M) = (3, 3, 1, 1)$ denote the ingredient weight vector.

Charlie considers four scoring functions where $i \in \{C, H, K, M\}$:

- $\textbf{Score}_A(D) = \sum_i d_i \cdot w_i$
- $\textbf{Score}_B(D) = \sum_i \min(1, d_i) \cdot \min(1, w_i)$

- $\textbf{Score}_C(D) = \left(\sum_i d_i \cdot w_i\right) - \left(\sum_i d_i\right)$
- $\textbf{Score}_D(D) = \dfrac{\sum_i d_i \cdot w_i}{1 + \sum_i d_i}$

**Output**

Which scoring function(s) achieve the highest Precision@3 when retrieving the top 3 results? (Ties are broken by numeric Chocolate ID in ascending order.)

**Options:**

1. $\text{Score}_A$ only.
2. $\text{Score}_B$ only.
3. $\text{Score}_C$ only.
4. $\text{Score}_D$ only.
5. Both $\text{Score}_A$ and $\text{Score}_C$.
6. Both $\text{Score}_A$ and $\text{Score}_D$.

**Q5: Position-Based Inspection Effects [3 pts]**

Charlie uses $\text{Score}_A$ with weights $W = (3, 3, 1, 1)$ to produce a chocolate ranking (ties broken by numeric Chocolate ID, ascending). His probability of inspecting each position is:

$$p = [p_1, p_2, p_3, p_4, p_5] = [0.8, 0.6, 0.4, 0.2, 0.1]$$

**Output**

Compute $\lfloor 100 \times \text{PI-Recall@5} \rfloor$ under this inspection model. (Ties are broken by numeric Chocolate ID in ascending order.)

# Section B3

# Pattern Pursuit: Convolution in Signal Processing

**Background: Time Series and Convolution**

**Time Series:** A time series is an ordered sequence of numerical values measured over time, appearing in applications like weather forecasting, financial markets, health tracking, and sensor data. We denote a time series as $\mathbf{x} = [x_1, x_2, x_3, \ldots, x_n]$, where $n$ is the length.

**Kernel:** A kernel is a short sequence $\mathbf{h} = [h_1, h_2, \ldots, h_k]$ used to detect patterns in a time series. For example, $\mathbf{h} = [1, -1]$ detects changes between consecutive values.

**Convolution:** Convolution applies a kernel to a time series to extract features. The output is computed as:

$$y_i = \sum_{j=1}^{k} h_j \cdot x_{i+k-j}$$

where the output length is $m = n - k + 1$ (when no padding is used). The **aggregate outputs** $S$ and $S^{\text{abs}}$ are defined in terms of all convolution values:

$$S = \sum_{i=1}^{m} y_i; \quad S^{\text{abs}} = \sum_{i=1}^{m} |y_i|$$

**Computing Convolution (3 steps):**

1. Reverse the kernel
2. Slide the reversed kernel along the time series
3. At each position, multiply corresponding elements and sum

**Example:** Given $\mathbf{x} = [2, 4, 6, 8]$ and $\mathbf{h} = [1, -1]$:

| | |
|---|---|
| Reversed kernel: | $[-1, 1]$ |
| Position 1: | $(-1)(2) + (1)(4) = 2$ |
| Position 2: | $(-1)(4) + (1)(6) = 2$ |
| Position 3: | $(-1)(6) + (1)(8) = 2$ |
| Output: | $\mathbf{y} = [2, 2, 2]$ |
| Aggregates: | $S = S^{\text{abs}} = 2 + 2 + 2 = 6$ |

**Padding:** Adding values to the ends of $\mathbf{x}$ before convolution. For example, padding $[3, 5, 7]$ with one zero on each side (i.e. padding of 1) gives $[0, 3, 5, 7, 0]$. Unless stated otherwise, assume **no padding**.

**Stride:** The step size when sliding the kernel. Stride = 1 (default) means moving one position at a time; stride = 2 means skipping every other position. Unless stated otherwise, assume **stride = 1**.

## Questions

**Q1: Convolution with Custom Kernel [1 pt]**

You are given the following time series and kernel:

$$\mathbf{x} = [1, 3, 2, 4], \quad \mathbf{h} = [2, 1]$$

**Task:** Apply convolution with the following parameters:

- Use padding of 1 with value **2** on both ends (prepend and append one copy of the value 2)
- Use stride = 2

**Output**

Select the correct convolution output **y** from the options below.

**Options:**

A. $[4, 7, 8]$

B. $[2, 7, 4]$

C. $[5, 8, 10]$

D. $[8, 7, 4]$

E. $[4, 11, 8]$

F. $[7, 4, 8]$

**Q2: Signal Smoothness vs. Fluctuations [2 pts]**

Consider two time series with the same minimum (2) and maximum (10) values:

$$\mathbf{x} = [2, 4, 6, 8, 10], \quad \mathbf{z} = [2, 6, 3, 10, 4]$$

Signal **x** is smooth (values increase steadily), while signal **z** fluctuates (values jump up and down). We wish to find kernels that distinguish between these two types of signals.
**Task:** Find kernel $\mathbf{h} = [h_1, h_2, h_3]$ such that the aggregate output $S = \sum_{i=1}^{m} y_i$ (stride = 1, no padding) satisfies:

- Constant signals $\rightarrow S = 0$
- Smooth signals like $\mathbf{x} \rightarrow |S|$ small
- Fluctuating signals like $\mathbf{z} \rightarrow |S|$ large

**Output**

Select all kernels that satisfy the above conditions. *(More than one answer may be correct.)*

**Options:**

A. $[1, 0, -1]$

B. $[2, -3, 2]$

C. $[1, 1, -2]$

D. $[1, -2, 1]$

E. $[0, -1, 1]$

F. $[-2, 4, -2]$

**Q3: Signal Acceleration [2 pts]**

In time series analysis, **acceleration** measures whether a signal is speeding up or slowing down. Consider two time series:

$$\mathbf{x} = [1, 5, 8, 10, 11], \quad \mathbf{w} = [1, 2, 4, 7, 11]$$

Signal **x decelerates**: differences between consecutive values decrease ($4 \to 3 \to 2 \to 1$). Signal **w accelerates**: differences between consecutive values increase ($1 \to 2 \to 3 \to 4$).
**Task:** Find kernel $\mathbf{h} = [h_1, h_2, h_3]$ such that the aggregate output $S = \sum_{i=1}^{m} y_i$ (stride = 1, no padding) satisfies:

- Signals with constant velocity (constant differences between consecutive values) $\to S = 0$
- Decelerating signals like $\mathbf{x} \to S < 0$
- Accelerating signals like $\mathbf{w} \to S > 0$

**Output**

Select all kernels that satisfy the above conditions. *(More than one answer may be correct.)*

**Options:**

A. $[-1, 0, 1]$

B. $[-1, 2, -1]$

C. $[1, -1, 0]$

D. $[0, 1, -2]$

E. $[1, -2, 1]$

F. $[3, -6, 3]$

**Q4: Spike Detection [2 pts]**

A **spike** is a value significantly larger than both its immediate neighbours. Consider two signals:

$$\mathbf{x} = [1, 2, 8, 2, 1, 2], \quad \mathbf{z} = [1, 3, 5, 7, 9, 11]$$

Signal **x** contains a spike (value 8 is much larger than its neighbours), while signal **z** is smooth (no spikes).
**Task:** Find kernel $\mathbf{h} = [h_1, h_2, h_3]$ such that $S^{\text{abs}} = \sum_{i=1}^{m} |y_i|$ (stride = 1, no padding) satisfies:

- Constant signals $\to S^{\text{abs}} = 0$
- Smooth signals like $\mathbf{z} \to S^{\text{abs}} = 0$
- Signals with spikes like $\mathbf{x} \to S^{\text{abs}}$ large

**Output**

Select all kernels that satisfy the above conditions. *(More than one answer may be correct.)*

**Options:**

A. $[1, 1, -2]$

B. $[-1, 0, 1]$

C. $[2, -3, 1]$

D. $[1, -1, 0]$

E. $[-1, 2, -1]$

F. $[1, -2, 1]$

**Q5: Symmetry Detection [3 pts]**

A signal is **symmetric** if it reads the same forwards and backwards. Consider the two time series:

$$\mathbf{x} = [2, 5, 5, 2], \quad \mathbf{z} = [1, 2, 3, -1]$$

Signal $\mathbf{x}$ is symmetric while $\mathbf{z}$ is not.
**Task:** For length-4 signals $\mathbf{x} = [x_1, x_2, x_3, x_4]$, find kernel $\mathbf{h} = [h_1, h_2, h_3, h_4]$ such that the convolution output $y$ (stride $= 1$, no padding) satisfies:

- Symmetric signals $\rightarrow y = 0$
- Asymmetric signals $\rightarrow y \neq 0$

**Output**

Select all kernels that correctly detect symmetry by outputting zero for symmetric signals. *(More than one answer may be correct.)*

**Options:**

A. $[1, 1, 1, -1]$

B. $[1, -1, 1, -1]$

C. $[-1, 2, 1, 1]$

D. $[2, 1, -1, -2]$

E. $[1, -1, -1, 0]$

F. $[1, 0, 0, -1]$

## Section B4

# Model Compression: Building Efficient Classifiers

**Background: Binary Classification with Linear Models**

In binary classification, each example $\mathbf{x}_i \in \mathbb{R}^d$ is assigned a label $y_i \in \{0, 1\}$. A linear classifier computes a score:

$$z_i = \mathbf{w}^\top \mathbf{x}_i + b$$

where $\mathbf{w} \in \mathbb{R}^d$ is the **weight vector**, $b \in \mathbb{R}$ is the **bias term** (intercept), and the prediction rule is:

$$\hat{y}_i = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i \leq 0 \end{cases}$$

The decision boundary is the hyperplane $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$. Note that the classification threshold is at exactly 0, with strict inequality: scores $> 0$ predict class 1, while scores $\leq 0$ predict class 0.

**Matrix Notation:** For a dataset with $n$ examples, we write $\mathbf{s} = \mathbf{X}\mathbf{w} + b\mathbf{1}$, where $\mathbf{X} \in \mathbb{R}^{n \times d}$ is the feature matrix (each row is one example), $\mathbf{s} \in \mathbb{R}^n$ is the score vector, and $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector. When $b = 0$, we have $\mathbf{s} = \mathbf{X}\mathbf{w}$.

**Accuracy:** Measures the fraction of correctly classified examples:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{n} \times 100\%$$

**Indexing Convention:** We use 1-based indexing throughout this problem. Features are indexed from 1 to $d$, and examples are indexed from 1 to $n$.

**Background: Feature Pruning and Bias Correction**

**Why Prune?**
In many applications, we simplify models by removing features. This:

- Reduces computational cost
- Decreases memory requirements
- Can improve interpretability
- May prevent overfitting

**Feature Pruning:** Pruning a feature $j$ means setting its weight to zero: $w_j \leftarrow 0$. If we prune a set of features $S \subseteq \{1, \ldots, d\}$, the score changes from:

$$z_i = \mathbf{w}^\top \mathbf{x}_i + b \quad \text{to:} \quad z_i^{(\text{pruned})} = \sum_{k \notin S} w_k x_{ik} + b$$

The removed contribution is:

$$r_i^{(S)} = \sum_{j \in S} w_j x_{ij}$$

Therefore:

$$z_i^{(\text{pruned})} = z_i - r_i^{(S)}$$

**Bias Correction:** After pruning, the average score across the dataset may shift. A common technique is **bias correction**: adjust the bias so that the mean score is preserved. The correction formula is:

$$b' = b + \mu_S$$

where $\mu_S = \frac{1}{n} \sum_{i=1}^{n} r_i^{(S)}$ is the average removed contribution. The corrected score becomes:

$$z_i^{(\text{corrected})} = z_i^{(\text{pruned})} + \mu_S = z_i - r_i^{(S)} + \mu_S$$

This reduces systematic drift and can improve post-pruning accuracy without retraining.

## Questions:

### Q1: Reverse Engineering the Weights [4 pts]

You are given a binary classification task with **6 examples** and **4 features**. Let $\mathbf{X} \in \mathbb{R}^{6 \times 4}$ be the feature matrix and $\mathbf{w} \in \mathbb{R}^4$ be the weight vector (with no bias term, i.e., $b = 0$). The score vector is $\mathbf{s} = \mathbf{X}\mathbf{w} \in \mathbb{R}^6$, and predictions follow the rule above.

**Dataset:**

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

You are given two possible score vectors $\mathbf{s}^{(A)}$ and $\mathbf{s}^{(B)}$. Each has **4 known values** and **2 unknown values** (marked with ?). All known score values are **sign-consistent** with $\mathbf{y}$ (i.e., positive when $y_i = 1$, negative when $y_i = 0$).

$$\mathbf{s}^{(A)} = \begin{bmatrix} 2 \\ ? \\ -1 \\ -1 \\ -1 \\ ? \end{bmatrix}, \quad \mathbf{s}^{(B)} = \begin{bmatrix} 3 \\ -5 \\ ? \\ -1 \\ ? \\ 1 \end{bmatrix}$$

**Task:**

1. Solve for the weight vector $\mathbf{w}$ from the equation $\mathbf{s} = \mathbf{X}\mathbf{w}$.
2. Use the solved weights to fill in the missing scores (?).
3. Compute predictions $\hat{\mathbf{y}}$ and calculate the accuracy.
4. Choose the option with higher accuracy.

**Output**

Let $\mathbf{w}^*$ be the weight vector from the option with higher accuracy. Compute the squared $\ell_2$ magnitude:

$$\|\mathbf{w}^*\|_2^2 = (w_1^*)^2 + (w_2^*)^2 + (w_3^*)^2 + (w_4^*)^2$$

Report your answer as $\lfloor \|\mathbf{w}^*\|_2^2 \rfloor$.

### Q2: Finding the Optimal Pruning Strategy [3 pts]

You are given a binary classification task with **7 examples** and **7 features**. The weight vector $\mathbf{w} \in \mathbb{R}^7$ is already trained (note: at least one weight equals 0).

**Dataset:**

$$\mathbf{X} = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 3 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & -1 & 0 & 0 & 6 \\ 0 & 0 & 0 & 0 & -1 & 0 & 7 \\ 0 & 0 & 0 & 0 & 0 & 1 & -6 \\ 1 & 1 & 1 & 1 & 1 & 5 & -9 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

**Task:** You must prune **exactly two features**. This means:

1. Choose two feature indices $\{j, k\}$ where $j \neq k$ and $j, k \in \{1, 2, 3, 4, 5, 6, 7\}$.
2. Set $w_j = w_k = 0$ (equivalently, remove those columns from $\mathbf{X}$).
3. Compute the pruned score: $\mathbf{s}^{(j,k)} = \mathbf{X}\mathbf{w}^{(j,k)}$.

**Output**

Find the pair $\{j^*, k^*\}$ that maximizes the accuracy after pruning (without bias correction). Report the single integer $j^* + k^*$ (the sum of the two feature indices). Use 1-indexing, i.e., $j^*, k^* \in \{1, 2, 3, 4, 5, 6, 7\}$.

**Q3: Bias Correction for Improved Performance [3 pts]**

Use the best pruning pair $\{j^*, k^*\}$ from **Q2** and the corresponding pruned weight vector $\mathbf{w}^{(j^*,k^*)}$ (where $w_{j^*} = w_{k^*} = 0$ and other weights unchanged). The pruned score vector is $\mathbf{s}^{(j^*,k^*)} = \mathbf{X}\mathbf{w}^{(j^*,k^*)} \in \mathbb{R}^7$.
**Task:** You can now add an **integer bias term** $b$ to every score:

$$\mathbf{s}_{\text{bias}}(b) = \mathbf{s}^{(j^*,k^*)} + b\mathbf{1},$$

where $\mathbf{1} \in \mathbb{R}^7$ is the all-ones vector and $b \in \{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$. Predictions are made using the standard rule with threshold at 0.

1. Compute the accuracy for each allowed value of $b$.
2. Let $b^*$ be a value that maximizes accuracy.
3. **Tie-breaking rule:** If multiple values achieve the same maximum accuracy, choose the $b$ with the largest value (most positive); if still tied, choose the one with the smallest $|b|$.

**Output**

Report $\lfloor (j^* + k^*) \cdot |b^*| \rfloor$, where $j^*$ and $k^*$ are the feature indices from Q2. Use 1-indexing, i.e., $j^*, k^* \in \{1, 2, 3, 4, 5, 6, 7\}$.